

Лабораторная работа №1

Тема: «Применение хэширование для ассоциативного поиска по ключам»

Цель работы: Изучить основные принципы хэширования, построить программную модель, обеспечивающую формирование хэш-таблиц и ассоциативный поиск по ключам.

1. Формирование хэш-таблиц

Для эффективной и безотказной работы метода хэширования необходимо очень тщательно подходить как к изучению задачи на этапе ее постановки, так и к возможности использования конкретного алгоритма хэширования в контексте этой задачи. Так, на стадии изучения постановки задачи, в которой для доступа к табличным данным планируется использовать хэширование, требуется проводить тщательные исследования по направлениям: диапазон допустимых ключей, максимальное количество элементов в таблице, вероятность возникновения коллизий и т. д. При этом нужно знать как общие проблемы метода хэширования, так и частные проблемы конкретных алгоритмов хэширования.

1.1. Размещение записей с использованием хэширования

Как в любом другом способе организации структур хранения, логические записи группируются в физические записи (блоки) по k штук. Однако в отличие от всех других способов организации структур хранения здесь выбран особенный способ группировки. Определенным образом выбирается так называемая хэш-функция f . Аргументом этой функции является значение x первичного ключа логической записи. Тогда $f(x)$ указывает адрес расположения блока, в котором должна находиться логическая запись со значением ключа x . Функция f должна, по возможности, равномерно распределять значения x по физическим блокам. Обсуждению возможных хэш-функций посвящено достаточно много литературы, поэтому здесь мы не будем касаться этого вопроса. Можно лишь добавить, что иногда, исходя из специфики множества значений x первичного ключа, можно построить функцию f , удовлетворяющую всем необходимым условиям. Таким образом, логическая запись таблицы со значением x первичного ключа размещается в блоке внешней памяти по адресу $f(x)$. В этом блоке может находиться не более k записей. Может оказаться, что выбранная функция отображает в один адрес памяти (один блок) более k записей. Возникает так называемая коллизия, которая требует дополнительной обработки.

1.2. Поиск записи с заданным значением ключа и чтение

По заданному значению ключа x подсчитывается значение функции $f(x)$. Далее из таблицы считывается блок, находящийся по адресу $f(x)$. Внутри этого блока перебором ищется нужная запись. Если записей в блоке нет, то запись ищется согласно методу обработки коллизий. Число обращений к таблице при этом равно:

- единице, если запись находится в блоке;
- единице плюс некоторое число записей согласно методу обработки коллизий (как правило, небольшое число).

1.3. Модификации записи

Осуществляется поиск и чтение записи, затем в модифицируются поля записи (не являющиеся первичным ключом), запись заносится на свое место. Если модифицируется значение ключа, то занесение записи осуществляется как ввод новой записи (добавление).

1.4. Удаление записи

Осуществляется поиск и чтение записи. На место удаляемой записи заносится «пустая» запись (или признак «пустой» записи). Если удаляемая запись находилась в списке области переполнения (метод цепочек), удаление ее производится по правилам удаления элемента списка.

1.5. Добавление записи

При добавлении записи со значением ключа x подсчитывается адрес соответствующего блока $f(x)$. Если в блоке есть место, запись заносится в блок, блок записывается по своему адресу. Если блок заполнен, то поиск места для записи производится согласно методу обработки коллизий.

2. Выбор алгоритма хэширования

Программист может либо применить один из известных алгоритмов хэширования (что, по сути, означает использование определенной хэш-функции), либо изобрести свой алгоритм, наиболее точно отображающий специфику конкретной задачи. При этом необходимо понимать, что разработка хэш-функции происходит в два этапа:

1. Выбор способа перевода ключевых слов в числовую форму.
2. Выбор алгоритма преобразования числовых значений в набор хэш-адресов.

2.1. Выбор способа перевода ключевых слов в числовую форму

Вся информация, вводимая в компьютер, кодируется в соответствии с одной из систем кодирования (таблиц кодировки). В большинстве таких систем символы (цифры, буквы, служебные знаки) представляются однобайтовыми двоичными числами. Как правило, ключевые поля элементов таблиц — строки символов, поэтому наиболее простым является работа с ASCII значениями.

2.1.1. XOR со сдвигом

Одним из простейших примеров хэш-функции является побитный XOR каждого блока (ASCII значение в двоичной системе счисления). Часто при использовании подобного метода для каждого блока выполняется однобитный циклический сдвиг после вычисления хэш-кода. Это можно описать следующим образом.

- Установить n-битный хэш-код в ноль.
- Для каждого n-битного блока данных выполнить следующие операции:
 - сдвинуть циклически текущий хэш-код влево на один бит;
 - выполнить операцию XOR для очередного блока и хэш-кода.

Это даст эффект "случайности" входа и уничтожит любую регулярность, которая присутствует во входных значениях.

2.1.2. Аддитивный метод

Данный метод заключается в применении операции сложения к десятичным ASCII значениям символов строки ключевого поля. При этом полученная сумма может использоваться для дальнейших вычислений в полном объеме, либо только последние несколько цифр, при этом первые цифры просто отбрасываются.

2.1.3. Мультипликативный метод

Принцип тот же, что и в аддитивном методе, только вместо операции сложения используется операция умножения.

2.2. Выбор алгоритма преобразования числовых значений в набор хэш-адресов

Наиболее известные алгоритмы закрытого хэширования основаны на следующих методах:

- деления;
- умножения;
- квадрата;

2.2.1. Метод деления

Этот простой алгоритм закрытого хэширования основан на использовании остатка деления значения ключа K на число, равное или близкое к числу элементов таблицы M:

$$A(K) = K \bmod M \quad (2.1)$$

В результате деления образуется целый остаток A(K), который и принимается за индекс блока в таблице. Чтобы получить конечный адрес в памяти, нужно полученный индекс умножить на размер элемента в таблице. Для уменьшения коллизий необходимо соблюдать ряд условий:

- Значение M выбирается равным простому числу.
- Значение M не должно являться степенью основания, по которому производится перевод ключей в числовую форму. Так, для алфавита, состоящего из первых пяти английских букв и пробела {a,b,c,d,e,''}, основание системы равно 6. Исходя из этого, число элементов таблицы M не должно быть степенью 6^p.

2.2.2. Метод умножения

Вычисление хэш-адреса происходит в два этапа:

1. Вычисление нормализованного хэш-адреса в интервале [0..1] по формуле:

$$F(K) = (C * K) \bmod 1 \quad (2.2)$$

где C — некоторая константа из интервала $[0..1]$, K — результат преобразования ключа в его числовое представление, $\text{mod } 1$ означает, что $F(K)$ является дробной частью произведения $C \cdot K$.

2. Конечный хэш-адрес $A(K)$ вычисляется по формуле $A(K) = [M \cdot F(K)]$, где M — размер хэш-таблицы, а скобки $[]$ означают целую часть результата умножения. Удобно рассматривать эти две формулы вместе:

$$A(K) = [M \cdot (C \cdot K) \bmod 1]. \quad (2.3)$$

Кнут в качестве значения C рекомендует использовать «золотое сечение» — величину, равную $(\sqrt{5}-1)/2 \approx 0.6180339887$.

2.2.3. Метод квадрата

В литературе часто упоминается метод квадрата как один из первых методов генерации последовательностей псевдослучайных чисел. При этом он непременно подвергается критике за плохое качество генерируемых последовательностей. Но для процесса хэширования это не является недостатком. Более того, в ряде случаев это наиболее предпочтительный алгоритм вычисления значения хэш-функции. Суть метода проста: значение ключа возводится в квадрат, после чего берется необходимое количество средних битов результата. Возможны варианты — при различной длине ключа биты берутся с разных позиций. Для принятия решения об использовании метода квадрата для вычисления хэш-функции необходимо провести статистический анализ возможных значений ключей. Если они часто содержат большое количество нулевых битов, то это означает, что распределение значений битов в средней части квадрата ключа недостаточно равномерно. В этом случае использование метода квадрата неэффективно.

3. Методы обработки коллизий

В ходе реализации хэширования возможны коллизии. С ненулевой вероятностью хэш-функция может выдать одно значение для разных значений ключа. Такая ситуация называется коллизией ключей и проявляется в том, что при попытке занести в хэш-таблицу запись с новым ключом мы наталкиваемся на то, что требуемый элемент массива уже занят. Одним из выходов является увеличение размера массива, образование новой хэш-функции и расстановка заново всех занятых элементов массива.

Для обработки коллизий используются две группы методов:

- закрытые — в качестве резервных используются ячейки самой хэш-таблицы;
- открытые — для хранения элементов с одинаковыми хэш-адресами используется отдельная область памяти.

Яркий пример открытых методов — метод цепочек, который сам по себе является самостоятельным методом хэширования. К закрытым методам относятся: линейное зондирование, двойное хэширование. Их основная идея — при возникновении коллизии попытаться отыскать в хэш-таблице свободную ячейку. Процедуру поиска свободной ячейки называют пробитом, или рехэшированием (вторичным хэшированием).

3.1. Метод цепочек

Это один из наиболее очевидных способов разрешения коллизий. Если по смыслу в элементах массива хранятся записи типа T , то в данном случае к записи добавляется поле типа ссылки на T . При возникновении коллизии по причине включения новой записи для нового элемента выделяется динамическая память, и он включается в конец линейного списка, который начинается от первичного элемента массива. Если коллизия возникает при поиске ключа, то список переполнения просматривается либо до момента нахождения требуемого ключа, либо до конца, что означает отсутствие искомой записи (рисунок 3.1).

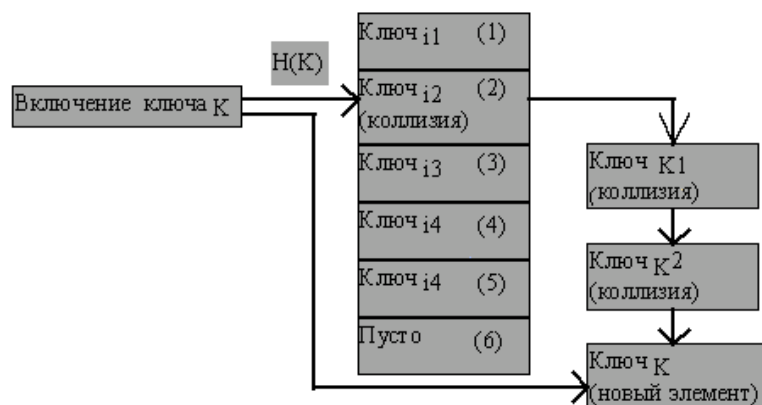


Рисунок 3.1

Метод цепочек переполнения легко реализуется, понятен, но потенциально приводит к излишним расходам памяти.

3.2. Линейное зондирование

Самым простым способом разрешения коллизий является использование метода линейного зондирования (иногда его называют методом линейных проб). Идея состоит в том, что сталкиваясь с коллизией при включении записи, мы последовательно (циклически, с переходом через конец на начало) просматриваем массив в поиске первого незанятого элемента. Если такой элемент обнаруживается, в него и заносится включаемая запись (рисунок 3.2). Если все элементы массива заняты, то это означает, что хэш-таблица переполнена и требуется расширение массива и новая расстановка его элементов.

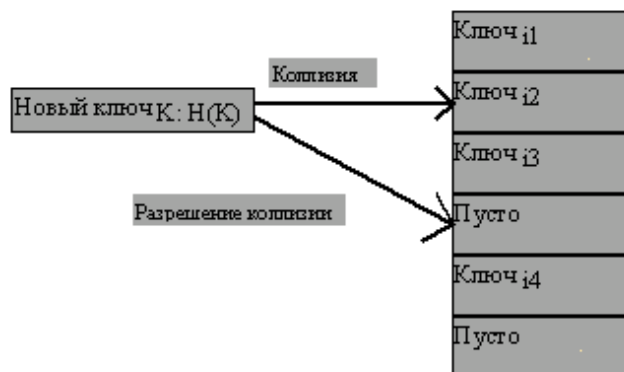


Рисунок 3.2

Если коллизия обнаруживается при поиске (т.е. следуя применяемой хэш-функции мы обращаемся по свертке ключа к элементу массива и обнаруживаем, что он занят записью с другим ключом), последовательно (циклически) просматриваются следующие элементы массива, пока не будет найдена запись с указанным ключом (рисунок 3.3), либо мы не наткнемся на свободный элемент массива. Последний вариант означает, что запись с указанным ключом в хэш-таблице отсутствует (рисунок 3.4).

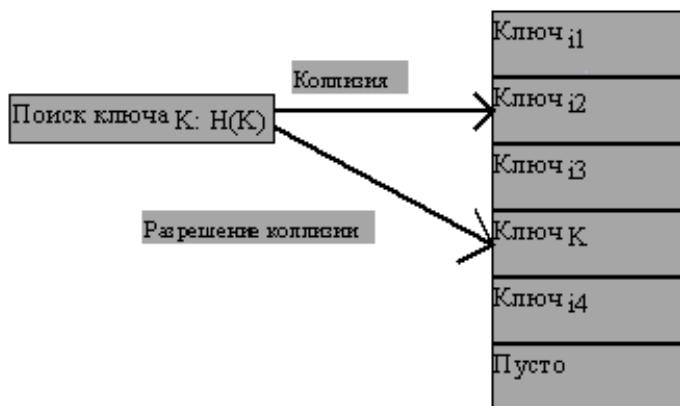


Рисунок 3.3

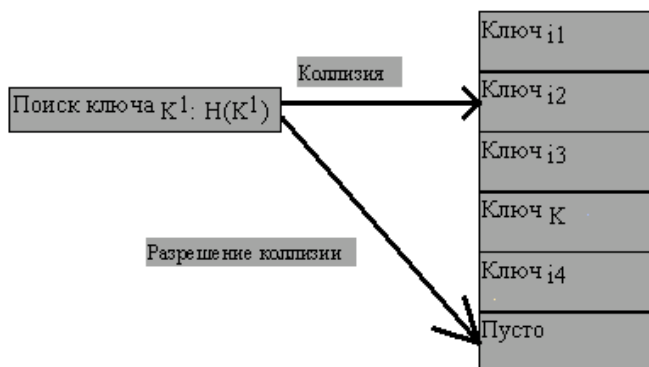


Рисунок 3.4

Достоинством метода линейного зондирования является его очевидная простота, а недостаток состоит в том, что при разрешении коллизии вторичные элементы массива (те, на которые прямо не указывает хэш-

функция) имеют обыкновение сосредотачиваться вблизи первичных элементов. Заполнение массива происходит неравномерно, и увеличивается вероятность следующих коллизий.

3.3. Двойное хэширование

Для обеспечения равномерного распределения ключей в хэш-таблице при наличии коллизий можно применять метод двойного хэширования. Он состоит в том, что если при включении или поиске ключа в хэш-таблице возникает коллизия, то к ключу (или к комбинации ключа и текущего индекса массива) применяется вторичная хэш-функция, значение которой указывает циклическое смещение в массиве от текущего индекса к элементу, в который следует включать или в котором следует искать требуемую запись (рисунки 3.5).

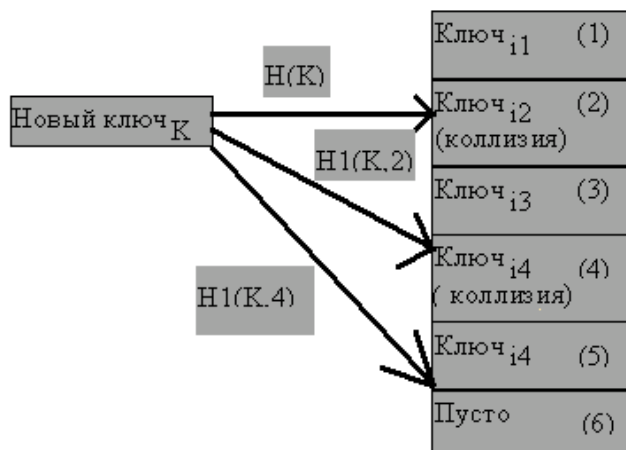


Рисунок 3.5

Поскольку метод двойного хэширования в полной форме приводит к слишком большим накладным расходам, на практике часто используется упрощенная форма этого метода, не гарантирующая равномерности записей в массиве, но обеспечивающая лучшие результаты, чем линейное зондирование. Один из способов состоит в использовании квадратичной функции при вычислении индекса следующей пробы. В этом случае последовательность вычисления индексов проб при включении или поиске записи является следующей:

$h_0 = H(K)$

 $h_i = (h_0 + i^2) \text{ MOD } N \quad (i > 0)$

Основным недостатком метода квадратичных проб является то, что для включаемой записи может не найтись свободного элемента массива даже в том случае, когда реально около половины элементов являются свободными.

Задание:

1) Написать на ЯВУ программу, осуществляющую просмотр, модификацию и сохранение таблицы записей. Реализовать добавление, удаление, редактирование записей и поиск по ключу перебором. Таблица должна содержать N строк (в таблицу заносится n записей) и не менее трех полей (с одним ключевым полем).

2) Написать на ЯВУ программу, которая преобразует таблицу данных из первого задания в хэш-таблицу на N строк (n записей). Для хэш-таблицы реализовать возможности добавления, удаления, модификации и поиска записей. Хэш-функция и метод обработки коллизий выбираются согласно варианту. Рассчитать вероятность возникновения коллизий (количество возникших коллизий при добавлении данных разделить на количество добавленных в хэш-таблицу записей) и среднее количество обращений к таблице во время поиска записи по ключу (сумма количества обращений к таблице при поиске каждой записи в хэш-таблице деленная на количество записей в хэш-таблице).

Варианты заданий.

Вариант	Тип хэш-функции	Метод обработки коллизий	Количество записей n	Объем хэш-таблицы N
	XOR со сдвигом + метод квадрата	линейное зондирование	30	32
	Аддитивный + метод деления	линейное зондирование	35	***
	Мультипликативный (1, 3, последний символы) + метод умножения	линейное зондирование	40	1.1*n
	Мультипликативный (все символы) 2 цифры + метод квадрата	линейное зондирование	40	64
	XOR без сдвига + метод квадрата	метод цепочек	20	16
	XOR без сдвига + метод умножения	метод цепочек	20	20
	Аддитивный + метод квадрата	метод цепочек	25	32
	Мультипликативный (все символы) 4 цифры + метод деления	метод цепочек	20	***
	XOR со сдвигом + метод деления	двойное хеширование	35	***
	XOR со сдвигом + метод умножения	двойное хеширование	40	1.1*n
	Аддитивный + метод квадрата	двойное хеширование	25	32
	Мультипликативный (1, 3, последний символы) 4 цифры + метод деления	двойное хеширование	30	***

*** - размер таблицы выбирается в соответствии с методом деления, то есть как ближайшее простое число больше n.

Ожидаемый вывод программы:

1. Консольное приложение
2. Вывод в консоль обычной и хэш-таблиц в виде таблиц (под конкретный язык найти соответствующую библиотеку и использовать). Для Python можно PrettyTable: `pip install PrettyTable`.
3. Используемая таблица должна быть предварительно заполненной согласно варианту.
4. В таблице должно быть не менее 3-х полей. Выбор поля для хеширования остается на усмотрение студента, но данное поле в обязательном порядке должно работать с символьным/строковым типом данных (не числовой тип). Пример таблицы: Название товара (хэш-поле) - Количество - Цена.

5. Добавить меню для взаимодействия с программой. Меню нумеровать от 1 до N. Переход к пунктам меню можно осуществлять через «Введите номер пункта меню: » или стрелочками вверх/вниз, подключив соответствующую библиотеку.

Пример:

1. Работа с таблицей
2. Работа с хэш-таблицей

Далее в каждой из таблиц есть свое подменю. Для обычной таблицы:

1. Вывести таблицу
2. Добавление
3. Редактирование
4. Удаление
5. Поиск

Для хэш-таблицы:

1. Вывести таблицу
2. Добавление
3. Редактирование
4. Удаление
5. Поиск
6. Коллизии

6. Для программы реализовать функции добавления, редактирования, удаления и поиска информации.

1. Пример добавления:

Поле_1: данные_1

Поле_2: данные_2

...

Поле_N: данные_N

—

Данные успешно добавлены

Продолжить добавление: yes/no (или Enter/Esc, можно другие варианты)?

2. Пример поиска:

Поиск «Поле_1»: данные_1

Затем необходимо выполнить поиск данных и, если они существуют - вывести информацию о запрошенной строке таблицы. Если не существуют - сообщить об этом и предложить повторить ввод или отказаться.

3. Пример удаления. Необходимо ввести поле по которому выполняется хэширование, например это Поле_1:

Поиск «Поле_1»: данные_1

Затем необходимо выполнить поиск данных и, если они существуют - удалить и вывести соответствующее сообщение. Если не существуют - сообщить об этом и предложить повторить ввод или отказаться.

4. Пример редактирования. Необходимо ввести поле по которому выполняется хэширование, например это Поле_1:

Поиск «Поле_1»: данные_1

Далее аналогично предыдущему подпункту №2.

Поле_1 (текущее значение) : «нажмите Enter чтобы оставить без изменений или напишите чтобы изменить значение»

...

—

Данные успешно обновлены

7. Добавить в меню вывод текущего состояния коллизий: сколько всего записей, сколько было коллизий. Число коллизий изменяется может измениться только при добавлении новой информации.

8. Построить график зависимости числа добавленных записей от числа коллизий. По оси X число записей, по оси Y текущее суммарное число коллизий.

9. При взаимодействии с программой консоль принудительно не очищать

Содержание отчета:

1. Титульный лист

2. Исходная таблица данных
3. Хэш-таблица с пометками мест возникновения коллизий
4. Расчетные данные
5. Вывод